

Tugas 4

Struktur Data

Binary Search Tree



Dosen Pengampu :

M. Chandra Cahyo Utomo, S. Kom., M. Kom.

NIP. 199202202019031000

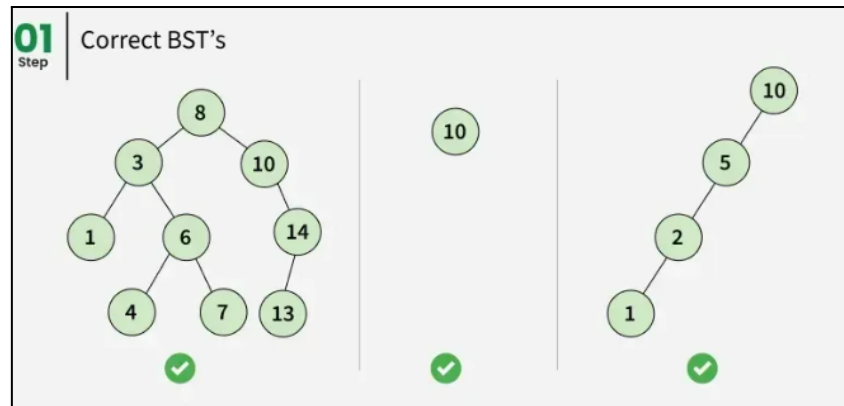
Disusun Oleh Kelompok 2 :

Cindy Aulia Renita	11241023
Fauzan Fayzul Haq	11241028
Thisya Darmala	11241083
Zharif Aziz Zulkarnain Widodo	11231092

30 Oktober 2025

Dasar Teori

Binary Search Tree adalah struktur data pohon biner di mana setiap node menyimpan kunci unik dan memenuhi aturan pengurutan: semua nilai di *subtree* kiri lebih kecil daripada nilai node, dan semua nilai di subtree kanan lebih besar daripada nilai node. Struktur ini bersifat rekursif: setiap *subtree* kiri atau kanan juga merupakan BST.

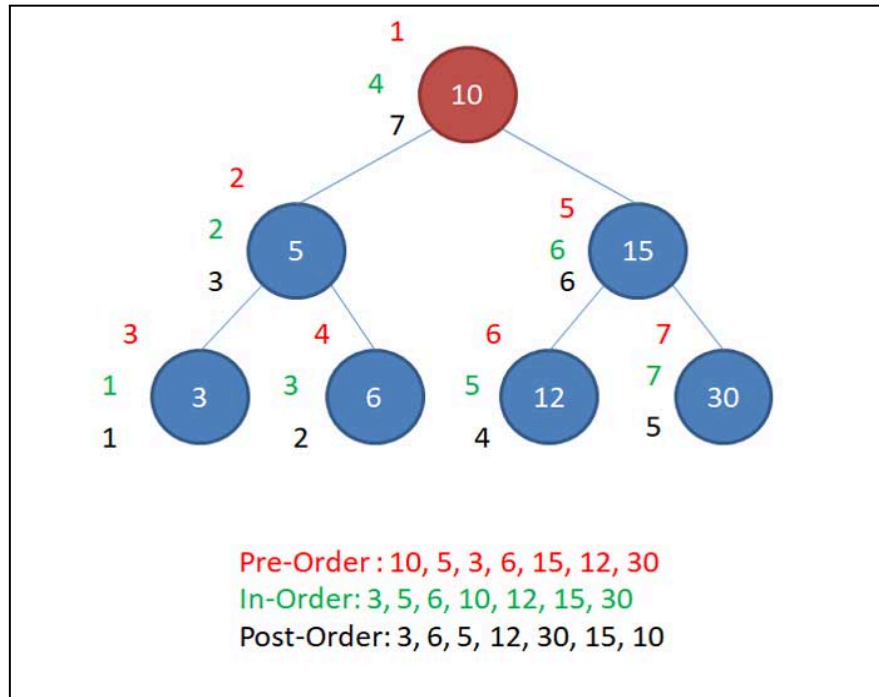


Operasi Dasar *Binary Search Tree* :

- 1) Pencarian/*Search*: mulai dari root, bandingkan kunci lalu bergerak ke kiri atau kanan sesuai perbandingan hingga ketemu atau mencapai daun.
- 2) Penyisipan/*Insert*: cari posisi daun yang sesuai menurut aturan urutan lalu tambahkan node baru.
- 3) Penghapusan/*Remove*: ada tiga kasus umum node daun, node dengan satu anak, dan node dengan dua anak (mengganti dengan inorder successor atau predecessor). Dalam BST seimbang, operasi ini umumnya $O(\log n)$; pada kasus terburuk (pohon sangat tidak seimbang) menjadi $O(n)$.

Tiga cara rekursif untuk mengunjungi semua node pada pohon biner :

- 1) *Pre-order*: memproses (mengunjungi) node saat ini terlebih dahulu lalu menelusuri subtree kiri dan subtree kanan (*Root* $\rightarrow$ *Left* $\rightarrow$ *Right*).
- 2) *In-order*: menelusuri subtree kiri dulu kemudian memproses node dan terakhir subtree kanan sehingga pada BST menghasilkan daftar nilai yang terurut naik (*Left* $\rightarrow$ *Root* $\rightarrow$ *Right*).
- 3) *Post-order*: mengunjungi subtree kiri dan kanan dahulu baru memproses node induk terakhir sehingga berguna bila anak harus diproses sebelum induk, misalnya saat penghapusan pohon atau menghitung ukuran subtree (*Left* $\rightarrow$ *Right* $\rightarrow$ *Root*).



Source Code

No	Node.java
1	<code>package Tugas_4;</code>
2	
3	<code>class Node {</code>
4	<code> char key;</code>
5	<code> Node left, right;</code>
6	
7	<code> public Node(char item) {</code>
8	<code> key = item;</code>
9	<code> left = right = null;</code>
10	<code> }</code>
	<code>}</code>

No	BinarySearchTree.java
1	<code>package Tugas_4;</code>
2	
3	<code>import java.util.ArrayList;</code>
4	
5	<code>class BinarySearchTree {</code>
6	<code> Node root;</code>
7	<code> private ArrayList<Character> insertOrder; // To track</code>
8	<code>insertion order (updated after insert/remove)</code>
9	
10	<code> BinarySearchTree() {</code>
11	<code> root = null;</code>
12	<code> insertOrder = new ArrayList<>();</code>
13	<code> }</code>
14	<code> // Insert a key</code>
15	<code> void insert(char key) {</code>
16	<code> Node newNode = insertRec(root, key);</code>
17	<code> if (newNode != null) { // Insert successful (not</code>
18	<code>duplicate)</code>
19	<code> insertOrder.add(key);</code>
20	<code> }</code>
21	<code> root = newNode != null ? newNode : root; // Update</code>
22	<code>root if changed</code>
23	<code> }</code>
24	
25	<code> Node insertRec(Node root, char key) {</code>
26	<code> if (root == null) {</code>
27	<code> return new Node(key);</code>
28	<code> }</code>
29	<code> if (key < root.key)</code>
	<code> root.left = insertRec(root.left, key);</code>

```

30         else if (key > root.key)
31             root.right = insertRec(root.right, key);
32         // If key == root.key, do nothing (no duplicates)
33         return root;
34     }
35
36     // Remove a key
37     void remove(char key) {
38         root = removeRec(root, key);
39         insertOrder.remove(Character.valueOf(key)); //
40         // Remove from insertion order list if exists
41     }
42
43     Node removeRec(Node root, char key) {
44         if (root == null) return root;
45
46         if (key < root.key)
47             root.left = removeRec(root.left, key);
48         else if (key > root.key)
49             root.right = removeRec(root.right, key);
50         else {
51             // Node with only one child or no child
52             if (root.left == null) return root.right;
53             else if (root.right == null) return root.left;
54
55             // Node with two children: Get the inorder
56             // successor
57             root.key = minValue(root.right);
58
59             // Delete the inorder successor
60             root.right = removeRec(root.right, root.key);
61         }
62         return root;
63     }
64
65     char minValue(Node root) {
66         char minv = root.key;
67         while (root.left != null) {
68             minv = root.left.key;
69             root = root.left;
70         }
71         return minv;
72     }
73
74     // Pre-order traversal
75     void preOrder() {
76         preOrderRec(root);
77     }
78
79     void preOrderRec(Node root) {

```

```

75         if (root != null) {
76             System.out.print(root.key + " ");
77             preOrderRec(root.left);
78             preOrderRec(root.right);
79         }
80     }
81
82     // In-order traversal
83     void inOrder() {
84         inOrderRec(root);
85     }
86
87     void inOrderRec(Node root) {
88         if (root != null) {
89             inOrderRec(root.left);
90             System.out.print(root.key + " ");
91             inOrderRec(root.right);
92         }
93     }
94
95     // Post-order traversal
96     void postOrder() {
97         postOrderRec(root);
98     }
99
100    void postOrderRec(Node root) {
101        if (root != null) {
102            postOrderRec(root.left);
103            postOrderRec(root.right);
104            System.out.print(root.key + " ");
105        }
106    }
107
108    // Get keys in insertion order (updated after
109    insert/remove)
110    public char[] getInsertOrder() {
111        char[] insertArray = new char[insertOrder.size()];
112        for (int i = 0; i < insertOrder.size(); i++) {
113            insertArray[i] = insertOrder.get(i);
114        }
115        return insertArray;
116    }
117
118    // Visualize the tree with ASCII art and colors (using
119    ANSI escape codes)
120    public void visualize() {
121        System.out.println("Visualisasi Tree (horizontal,
122        root di kiri, cabang kanan atas dengan /, cabang kiri bawah
123        dengan \\):");
124        if (root == null) {

```

```

122         System.out.println("Tree kosong.");
123         return;
124     }
125     printTree(root, "", true);
126     System.out.print("\u001B[0m"); // Reset color at the
127 end
128     }
129
130     private void printTree(Node node, String indent, boolean
131 last) {
132         if (node != null) {
133             System.out.print(indent);
134             if (last) {
135 System.out.print("\u001B[34mR-----\u001B[0m"); // Blue for
136 right branch
137                 indent += "        ";
138             } else {
139
140 System.out.print("\u001B[31mL-----\u001B[0m"); // Red for
141 left branch
142                 indent += "|        ";
143             }
144             System.out.println("\u001B[32m" + node.key +
145 "\u001B[0m"); // Green for node key
146             printTree(node.left, indent, false);
147             printTree(node.right, indent, true);
148         }
149     }
150 }

```

No	Main.java
1	<code>package Tugas_4;</code>
2	
3	<code>import java.util.Random;</code>
4	<code>import java.util.Scanner;</code>
5	
6	<code>public class Main {</code>
7	<code>public static void main(String[] args) {</code>
8	<code>BinarySearchTree tree = new BinarySearchTree();</code>
9	<code>Scanner scanner = new Scanner(System.in);</code>
10	<code>Random rand = new Random();</code>
11	<code>boolean running = true;</code>
12	<code>while (running) {</code>
13	<code>System.out.println("\nMenu:");</code>
14	

```

15         System.out.println("1. Insert data (huruf)");
16         System.out.println("2. Remove data (huruf)");
17         System.out.println("3. Generate 10 huruf random
18 dan insert");
19         System.out.println("4. Tampilkan Traversals ");
20         System.out.println("5. Visualisasi Tree");
21         System.out.println("6. Keluar");
22         System.out.print("Pilih opsi: ");
23         int choice = scanner.nextInt();
24         scanner.nextLine(); // Konsumsi newline
25
26         switch (choice) {
27             case 1:
28                 System.out.print("Masukkan huruf untuk
29 insert: ");
30                 char insertKey =
31 scanner.nextLine().charAt(0);
32                 tree.insert(insertKey);
33                 System.out.println("Huruf '" + insertKey
34 + "' telah diinsert.");
35                 // Tampilkan array setelah insert
36                 char[] insertOrderArrayAfterInsert =
37 tree.getInsertOrder();
38                 System.out.println("Array setelah
39 insert:");
40                 for (char key :
41 insertOrderArrayAfterInsert) {
42                     System.out.print(key + " ");
43                 }
44                 System.out.println();
45                 if (insertOrderArrayAfterInsert.length
46 == 0) {
47                     System.out.println("Tidak ada
48 keys.");
49                 }
50                 break;
51             case 2:
52                 System.out.print("Masukkan huruf untuk
53 remove: ");
54                 char removeKey =
55 scanner.nextLine().charAt(0);
56                 tree.remove(removeKey);
57                 System.out.println("Huruf '" + removeKey
58 + "' telah diremove jika ada.");
59                 // Tampilkan array setelah remove
60                 char[] insertOrderArrayAfterRemove =
61 tree.getInsertOrder();
62                 System.out.println("Array setelah
63 remove:");
64                 for (char key :

```

```

60 insertOrderArrayAfterRemove) {
61     System.out.print(key + " ");
62 }
63 System.out.println();
64 if (insertOrderArrayAfterRemove.length
65 == 0) {
66     System.out.println("Tidak ada
67 keys.");
68 }
69 break;
70 case 3:
71     System.out.println("Generate dan insert
72 10 huruf random (A-Z):");
73     for (int i = 0; i < 10; i++) {
74         char randomChar = (char)
75         (rand.nextInt(26) + 'A');
76         System.out.print(randomChar + " ");
77         tree.insert(randomChar);
78     }
79     System.out.println("\n10 huruf random
80 telah diinsert.");
81     // Tampilkan array setelah generate dan
82     insert
83     char[] insertOrderArrayAfterGenerate =
84     tree.getInsertOrder();
85     System.out.println("Array setelah
86 generate dan insert:");
87     for (char key :
88     insertOrderArrayAfterGenerate) {
89         System.out.print(key + " ");
90     }
91     System.out.println();
92     if (insertOrderArrayAfterGenerate.length
93 == 0) {
94         System.out.println("Tidak ada
95 keys.");
96     }
97     break;
98 case 4:
99     System.out.println("Pre-order
100 traversal:");
101     tree.preOrder();
102     System.out.println("\nIn-order
103 traversal:");
104     tree.inOrder();
105     System.out.println("\nPost-order
106 traversal:");
107     tree.postOrder();
108     System.out.println();
109     break;

```

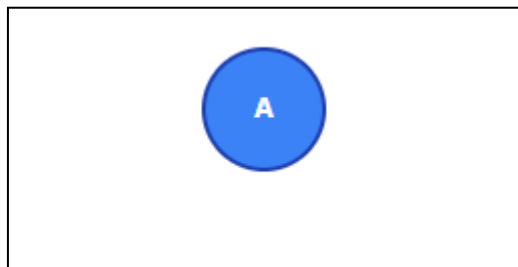
```
107         case 5:
108             tree.visualize();
109             break;
110         case 6:
111             running = false;
112             break;
113         default:
114             System.out.println("Opsi tidak valid.");
115     }
116 }
117 scanner.close();
118 }
```

Screenshot

1. Insert Data

Insert pada *Binary Search Tree* dilakukan dengan menempatkan nilai baru pada posisi yang menjaga properti pohon biner terurut yaitu semua nilai di kiri lebih kecil dan semua nilai di kanan lebih besar. Contoh pada gambar: memasukkan 'A' ke BST kosong menghasilkan satu node 'A' sebagai *root*.

```
Menu:  
1. Insert data (huruf)  
2. Remove data (huruf)  
3. Generate 10 huruf random dan insert  
4. Tampilkan Traversals  
5. Visualisasi Tree  
6. Keluar  
Pilih opsi: 1  
Masukkan huruf untuk insert: A  
Huruf 'A' telah diinsert.  
Array setelah insert:  
A
```



2. Remove Data

Remove pada *Binary Search Tree* mencari node dengan nilai yang ingin dihapus lalu menyesuaikan struktur pohon tanpa melanggar properti terurut. Seperti pada gambar ketika mencoba menghapus 'B' dari pohon yang hanya berisi 'A'. Operasi penghapusan umumnya dilakukan secara rekursif sehingga fungsi *remove* selalu mengembalikan pointer node yang diperbarui.

Menu:

1. Insert data (huruf)
2. Remove data (huruf)
3. Generate 10 huruf random dan insert
4. Tampilkan Traversals
5. Visualisasi Tree
6. Keluar

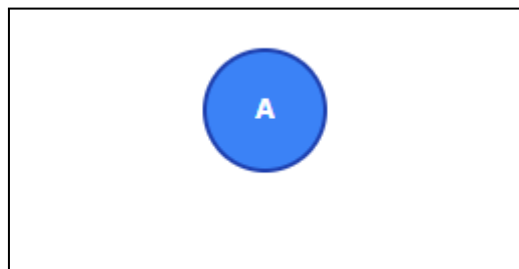
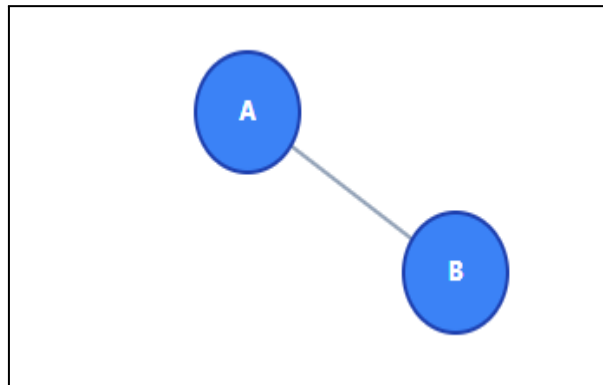
Pilih opsi: 2

Masukkan huruf untuk remove: B

Huruf 'B' telah diremove jika ada.

Array setelah remove:

A



3. Generate 10 Huruf Random dan *Insert*

Program menghasilkan 10 huruf kapital acak dari A–Z (R D R N I I V Z F F), menyisipkan masing-masing huruf tersebut ke dalam struktur data (ditampilkan sebagai *array* setelah proses), dan mengkonfirmasi bahwa 10 huruf telah di*insert*. Secara fungsional fitur pembuatan data acak dan penambahan ke koleksi, berguna untuk pengujian operasi selanjutnya seperti traversal atau visualisasi pohon.

```

Menu:
1. Insert data (huruf)
2. Remove data (huruf)
3. Generate 10 huruf random dan insert
4. Tampilkan Traversals
5. Visualisasi Tree
6. Keluar
Pilih opsi: 3
Generate dan insert 10 huruf random (A-Z):
R D R N I I V Z F F
10 huruf random telah diinsert.
Array setelah generate dan insert:
R D R N I I V Z F F

```

4. Tampilkan Transversal

Opsi ini akan menampilkan traversal dan program mencetak urutan Pre-order: R D N I F V Z, In-order: D F I N R V Z, dan Post-order: F I D N Z V R, ketiga urutan ini menunjukkan perbedaan cara pengunjungannya: *Pre-order* mengunjungi node sebelum anaknya (root dulu), *In-order* mengunjungi subtree kiri, lalu root, lalu subtree kanan sehingga menghasilkan urutan teratur alfabet jika struktur memang BST (dan di sini In-order menghasilkan urutan naik D F I N R V Z), sedangkan *Post-order* mengunjungi anak-anak sebelum root, berguna untuk operasi yang membutuhkan pemrosesan anak lebih dulu seperti penghapusan.

```

Menu:
1. Insert data (huruf)
2. Remove data (huruf)
3. Generate 10 huruf random dan insert
4. Tampilkan Traversals
5. Visualisasi Tree
6. Keluar
Pilih opsi: 4
Pre-order traversal:
R D N I F V Z
In-order traversal:
D F I N R V Z
Post-order traversal:
F I N D Z V R

```

5. Visualisasi Tree



Menu:

1. Insert data (huruf)
2. Remove data (huruf)
3. Generate 10 huruf random dan insert
4. Tampilkan Traversals
5. Visualisasi Tree
6. Keluar

Pilih opsi: 6

Process finished with exit code 0

Refrensi

GeeksforGeeks. (2025, 24 September). Binary Search Tree - GeeksforGeeks. Diakses 30 Oktober 2025, dari <https://www.geeksforgeeks.org/dsa/binary-search-tree-data-structure/>

SkillPlus. (2021, 11 Juli). Membuat Fungsi Traversal pada Binary Search Tree - SkillPlus. Diakses 30 Oktober 2025, dari <https://skillplus.web.id/membuat-fungsi-traversal-pada-binary-search-tree/>



Repository

https://github.com/Thisyath/Strukdat-smt-3/tree/master/src/Tugas_4